



Преподаватель:
Коляда
Никита Владимирович

Обратная связь:
• сообщения на inStudy

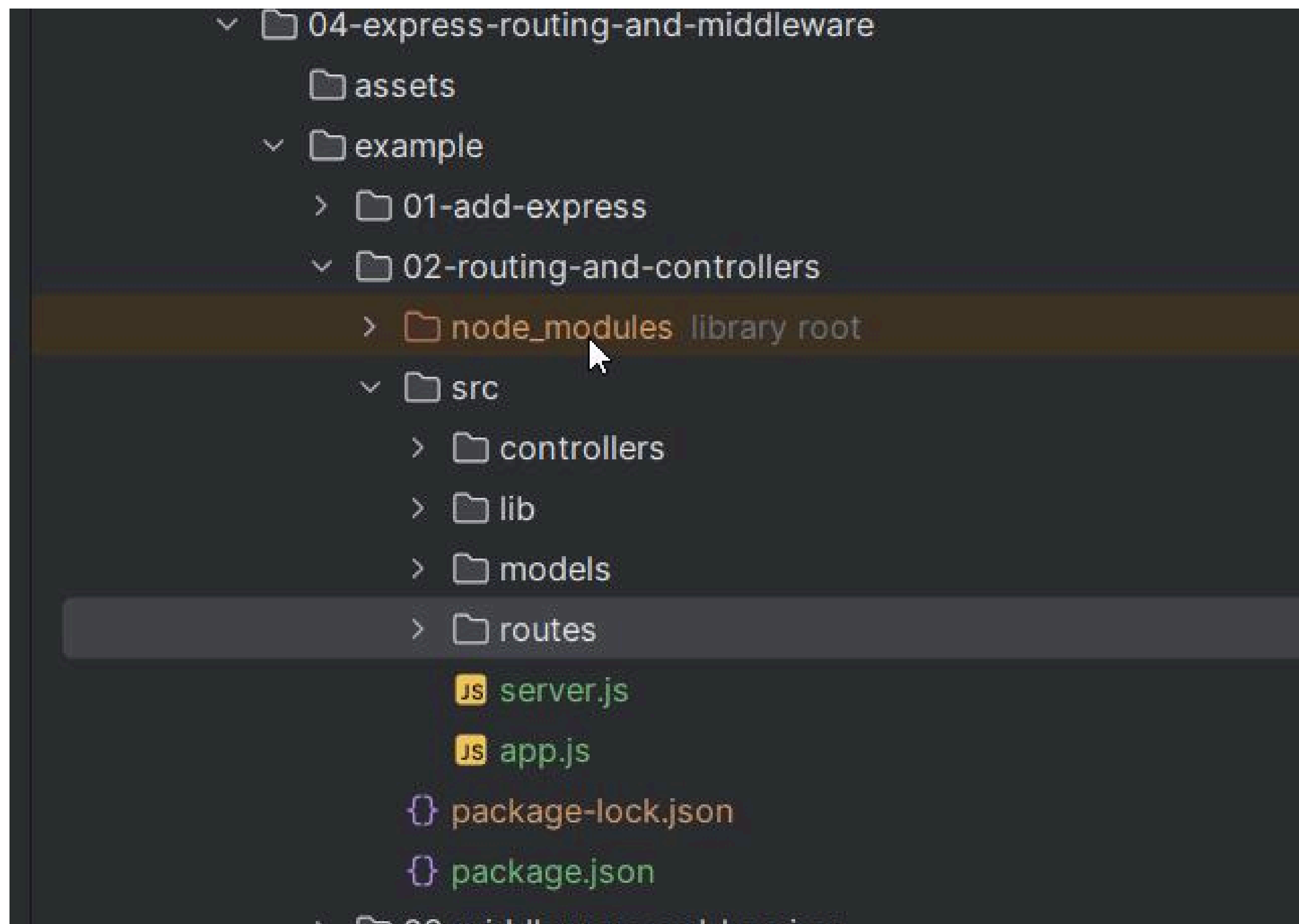
РЕАЛИЗАЦИЯ ПАТТЕРНА MVC – ПРАКТИКА

2026 Г.



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?



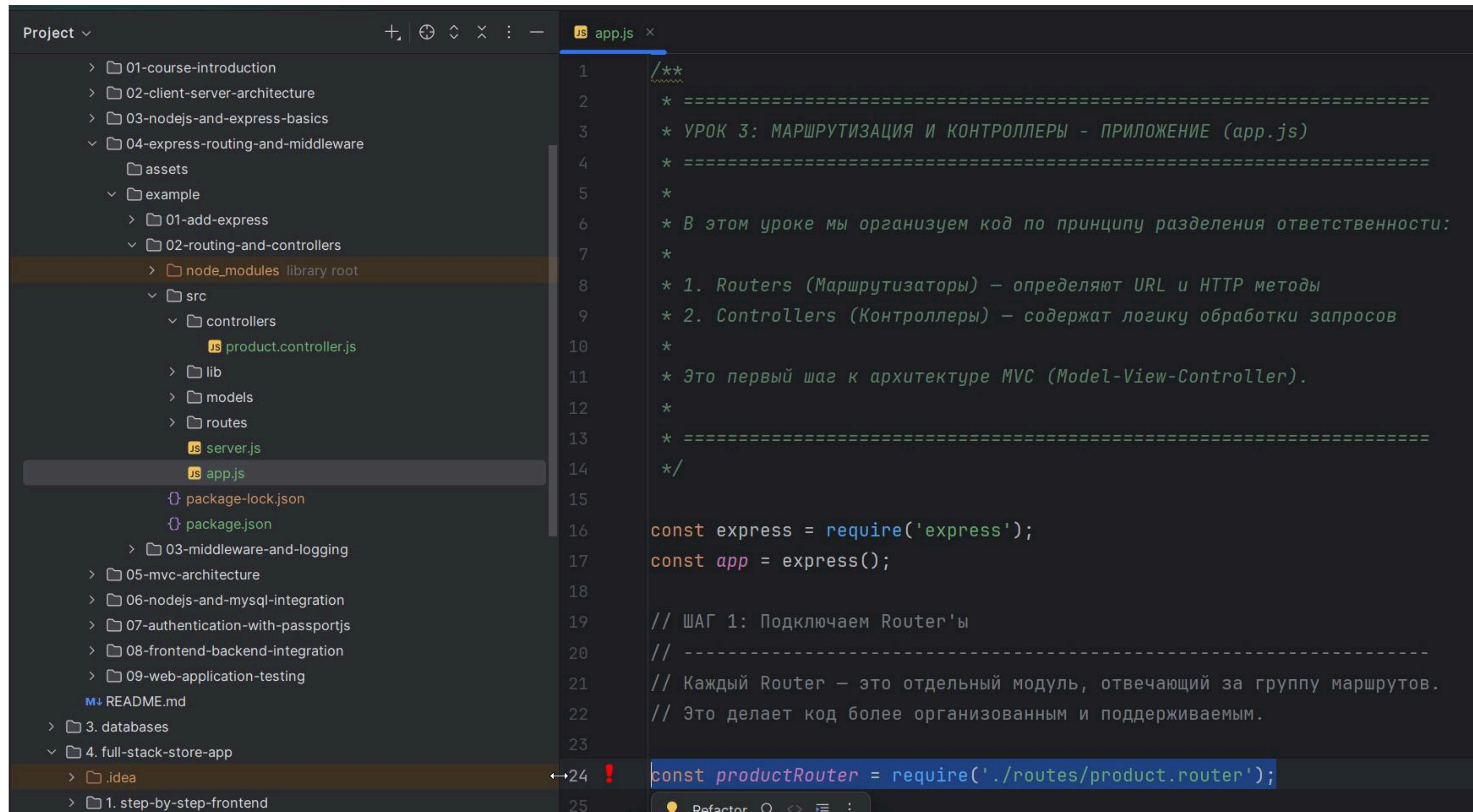
РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

-

ПРАКТИКА

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

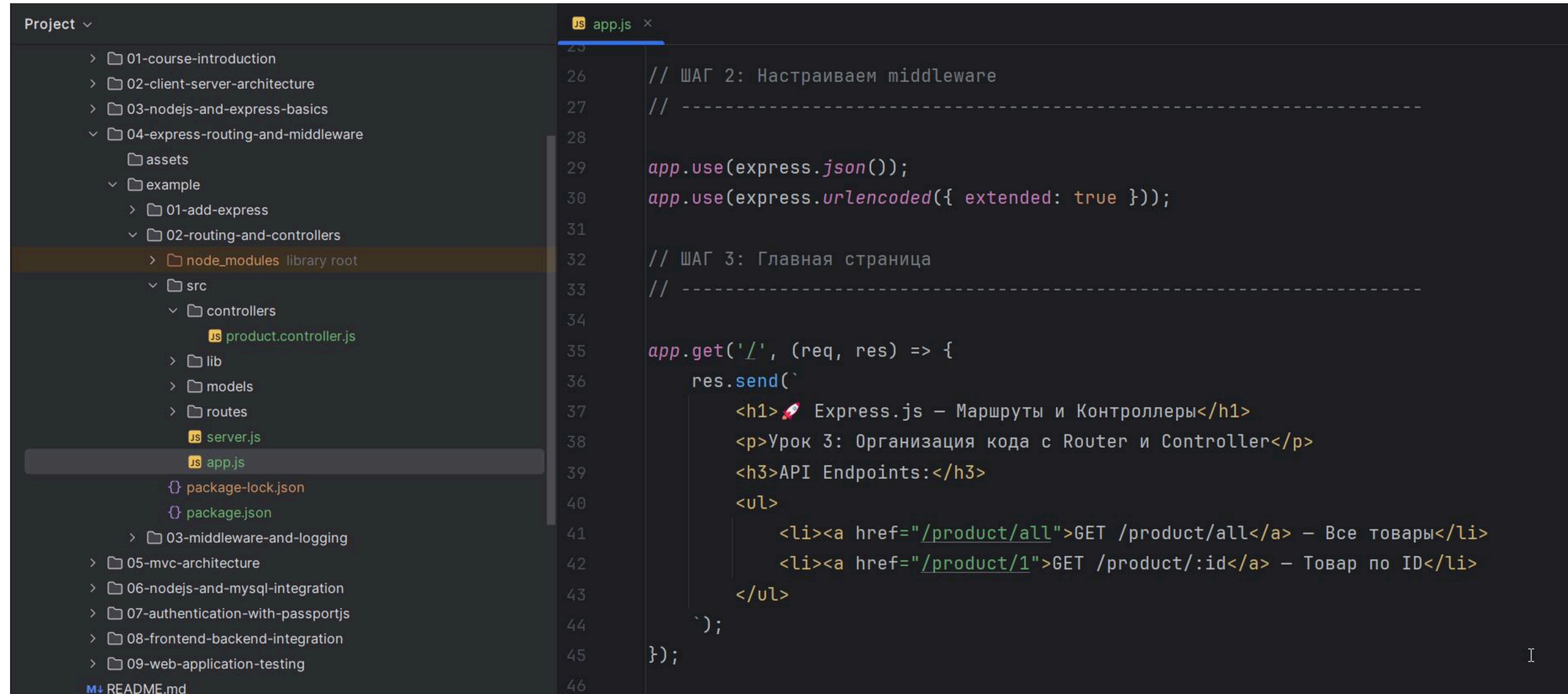


The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes folders for course introduction, client-server architecture, nodejs and express basics, express routing and middleware, and MVC architecture. The code editor shows the implementation of the MVC pattern in app.js.

```
1  /**
2  * =====
3  * УРОК 3: МАРШРУТИЗАЦИЯ И КОНТРОЛЛЕРЫ - ПРИЛОЖЕНИЕ (app.js)
4  * =====
5  *
6  * В этом уроке мы организуем код по принципу разделения ответственности:
7  *
8  * 1. Routers (Маршрутизаторы) – определяют URL и HTTP методы
9  * 2. Controllers (Контроллеры) – содержат логику обработки запросов
10 *
11 * Это первый шаг к архитектуре MVC (Model-View-Controller).
12 *
13 * =====
14 */
15
16 const express = require('express');
17 const app = express();
18
19 // ШАГ 1: Подключаем Router'ы
20 // -----
21 // Каждый Router – это отдельный модуль, отвечающий за группу маршрутов.
22 // Это делает код более организованным и поддерживаемым.
23
24 const productRouter = require('./routes/product.router');
```


РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

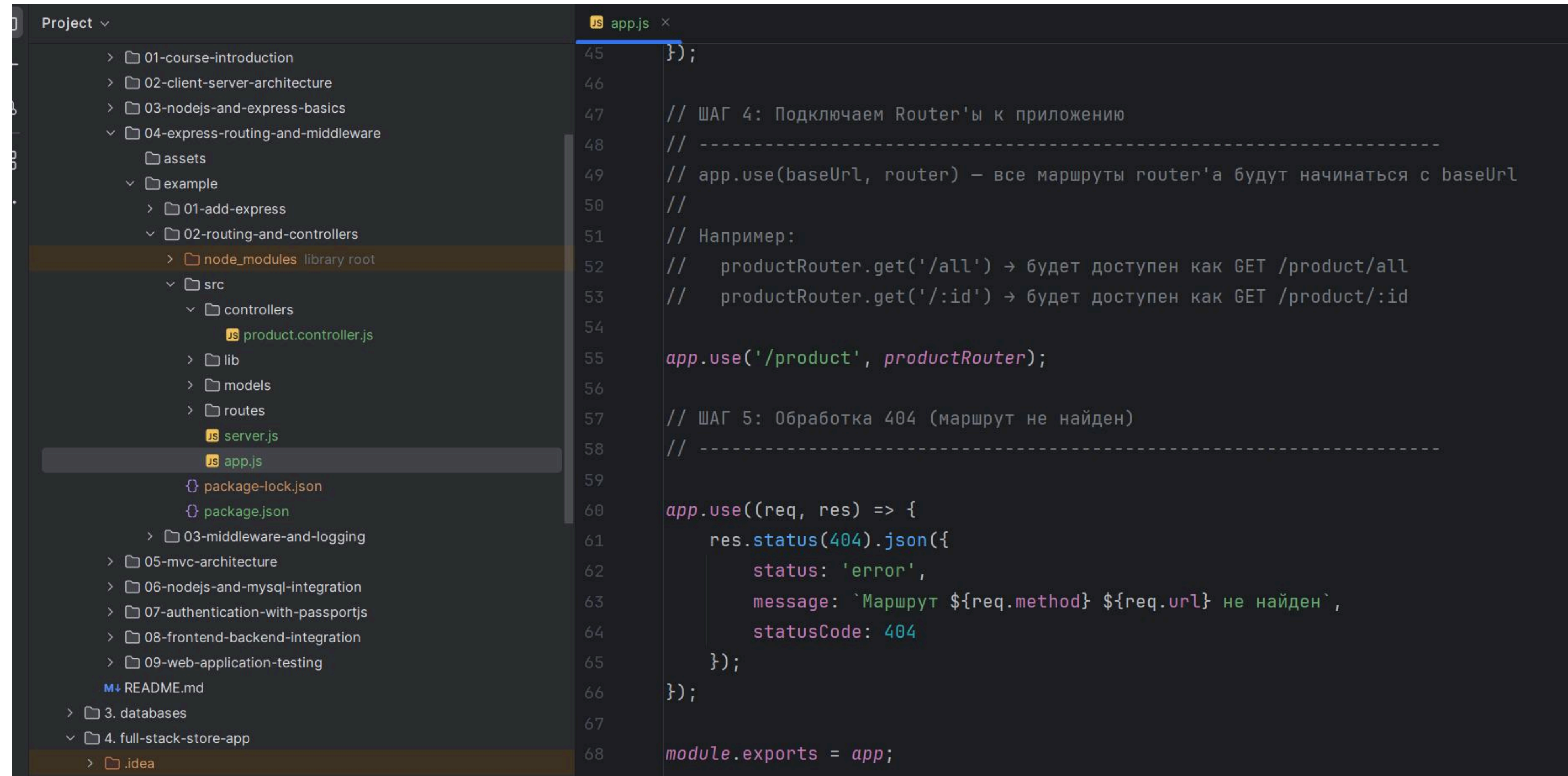


The screenshot shows a code editor with a project structure on the left and a JavaScript file named `app.js` open in the main editor. The project structure includes a folder named `04-express-routing-and-middleware` which contains subfolders `assets`, `example`, and `node_modules`. The `example` folder contains `01-add-express` and `02-routing-and-controllers`. The `node_modules` folder contains `src`, which in turn contains `controllers` (with `product.controller.js`), `lib`, `models`, `routes`, `server.js`, and `app.js`. The `app.js` file is highlighted in the project structure. The main editor shows the following code:

```
25
26 // ШАГ 2: Настраиваем middleware
27 // -----
28
29 app.use(express.json());
30 app.use(express.urlencoded({ extended: true }));
31
32 // ШАГ 3: Главная страница
33 // -----
34
35 app.get('/', (req, res) => {
36   res.send(`
37     <h1>🚀 Express.js – Маршруты и Контроллеры</h1>
38     <p>Урок 3: Организация кода с Router и Controller</p>
39     <h3>API Endpoints:</h3>
40     <ul>
41       <li><a href="/product/all">GET /product/all</a> – Все товары</li>
42       <li><a href="/product/1">GET /product/:id</a> – Товар по ID</li>
43     </ul>
44   `);
45 });
46
```


РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

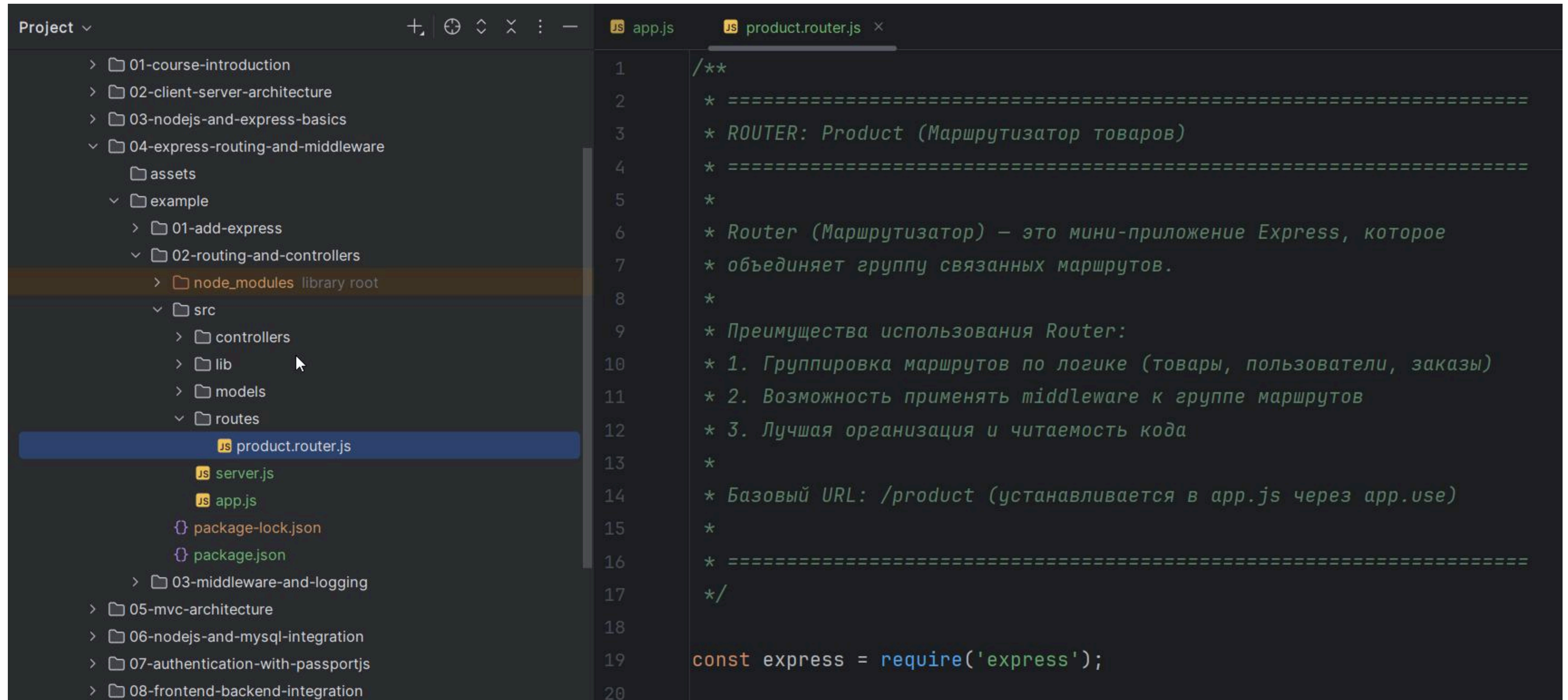


The screenshot shows an IDE with a project structure on the left and code in the main editor. The project structure includes folders for course introduction, client-server architecture, nodejs and express basics, express routing and middleware, MVC architecture, nodejs and mysql integration, authentication with passportjs, frontend-backend integration, web application testing, databases, and a full-stack store app. The code in the main editor is for `app.js` and implements the MVC pattern.

```
45  });
46
47  // ШАГ 4: Подключаем Router'ы к приложению
48  // -----
49  // app.use(baseUrl, router) – все маршруты router'a будут начинаться с baseUrl
50  //
51  // Например:
52  //   productRouter.get('/all') → будет доступен как GET /product/all
53  //   productRouter.get('/:id') → будет доступен как GET /product/:id
54
55  app.use('/product', productRouter);
56
57  // ШАГ 5: Обработка 404 (маршрут не найден)
58  // -----
59
60  app.use((req, res) => {
61    res.status(404).json({
62      status: 'error',
63      message: `Маршрут ${req.method} ${req.url} не найден`,
64      statusCode: 404
65    });
66  });
67
68  module.exports = app;
```


РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

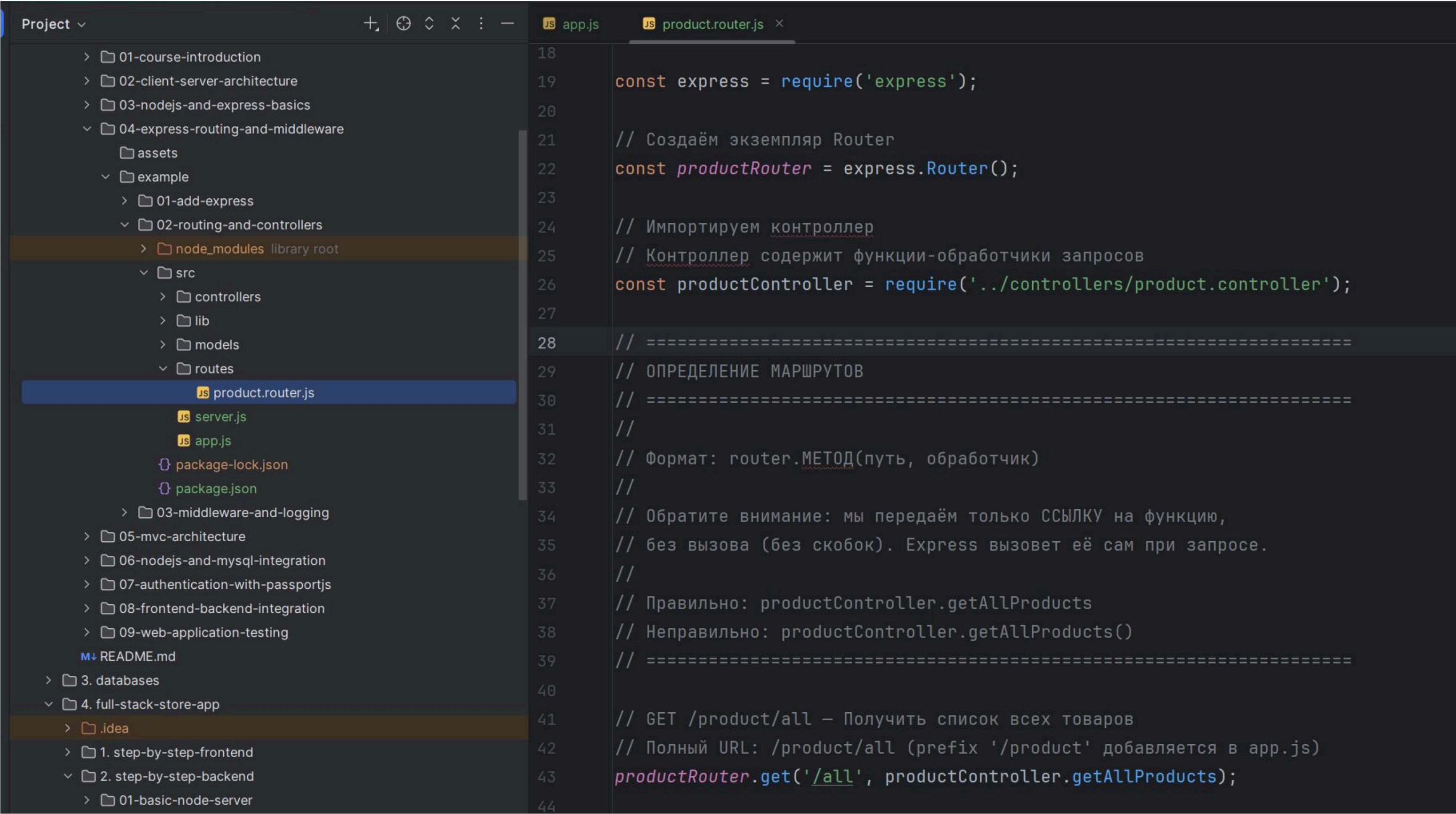
ПРАКТИКА



ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes folders for course introduction, client-server architecture, nodejs and express basics, and express routing and middleware. The 'express routing and middleware' folder is expanded, showing subfolders like 'assets', 'example', and 'src'. The 'src' folder is further expanded, showing 'controllers', 'lib', 'models', and 'routes'. The 'product.router.js' file is selected in the 'routes' folder. The code editor shows the implementation of the MVC pattern in JavaScript. It includes comments in Russian explaining the steps: creating an express instance, creating a Router, importing a controller, and defining routes. The code uses 'require' to load 'express' and 'product.controller'. It defines a 'productRouter' and a 'productController'. The route definition is commented out, showing the correct and incorrect ways to define a route. The final line shows the 'productRouter.get' method being used to define a route for '/all' that calls 'productController.getAllProducts'.

```
18
19 const express = require('express');
20
21 // Создаём экземпляр Router
22 const productRouter = express.Router();
23
24 // Импортируем контроллер
25 // Контроллер содержит функции-обработчики запросов
26 const productController = require('../controllers/product.controller');
27
28 // =====
29 // ОПРЕДЕЛЕНИЕ МАРШРУТОВ
30 // =====
31 //
32 // Формат: router.МЕТОД(путь, обработчик)
33 //
34 // Обратите внимание: мы передаём только ССЫЛКУ на функцию,
35 // без вызова (без скобок). Express вызовет её сам при запросе.
36 //
37 // Правильно: productController.getAllProducts
38 // Неправильно: productController.getAllProducts()
39 // =====
40
41 // GET /product/all – Получить список всех товаров
42 // Полный URL: /product/all (prefix '/product' добавляется в app.js)
43 productRouter.get('/all', productController.getAllProducts);
44
```


РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

01-course-introduction

02-client-server-architecture

03-nodejs-and-express-basics

04-express-routing-and-middleware

- assets
- example
 - 01-add-express
 - 02-routing-and-controllers
 - node_modules library root
 - src
 - controllers
 - lib
 - models
 - routes
 - product.router.js
 - server.js
 - app.js
 - package-lock.json
 - package.json
 - 03-middleware-and-logging
- 05-mvc-architecture
- 06-nodejs-and-mysql-integration
- 07-authentication-with-passportjs
- 08-frontend-backend-integration
- 09-web-application-testing

README.md

databases

full-stack-store-app

.idea

1_step-by-step-frontend

JS app.js

JS product.router.js

28

// =====

29

// ОПРЕДЕЛЕНИЕ МАРШРУТОВ

30

// =====

31

//

32

// Формат: router.МЕТОД(путь, обработчик)

33

//

34

// Обратите внимание: мы передаём только ССЫЛКУ на функцию,

35

// без вызова (без скобок). Express вызовет её сам при запросе.

36

//

37

// Правильно: productController.getAllProducts

38

// Неправильно: productController.getAllProducts()

39

// =====

40

41

// GET /product/all – Получить список всех товаров

42

// Полный URL: /product/all (prefix '/product' добавляется в app.js)

43

productRouter.get('/all', productController.getAllProducts);

44

45

// GET /product/:id – Получить один товар по его ID

46

// ":id" – это параметр маршрута, его значение доступно в req.params.id

47

// Пример: GET /product/5 → req.params.id === "5"

48

productRouter.get('/:id', productController.getProduct);

49

50

// Экспортируем Router для использования в app.js

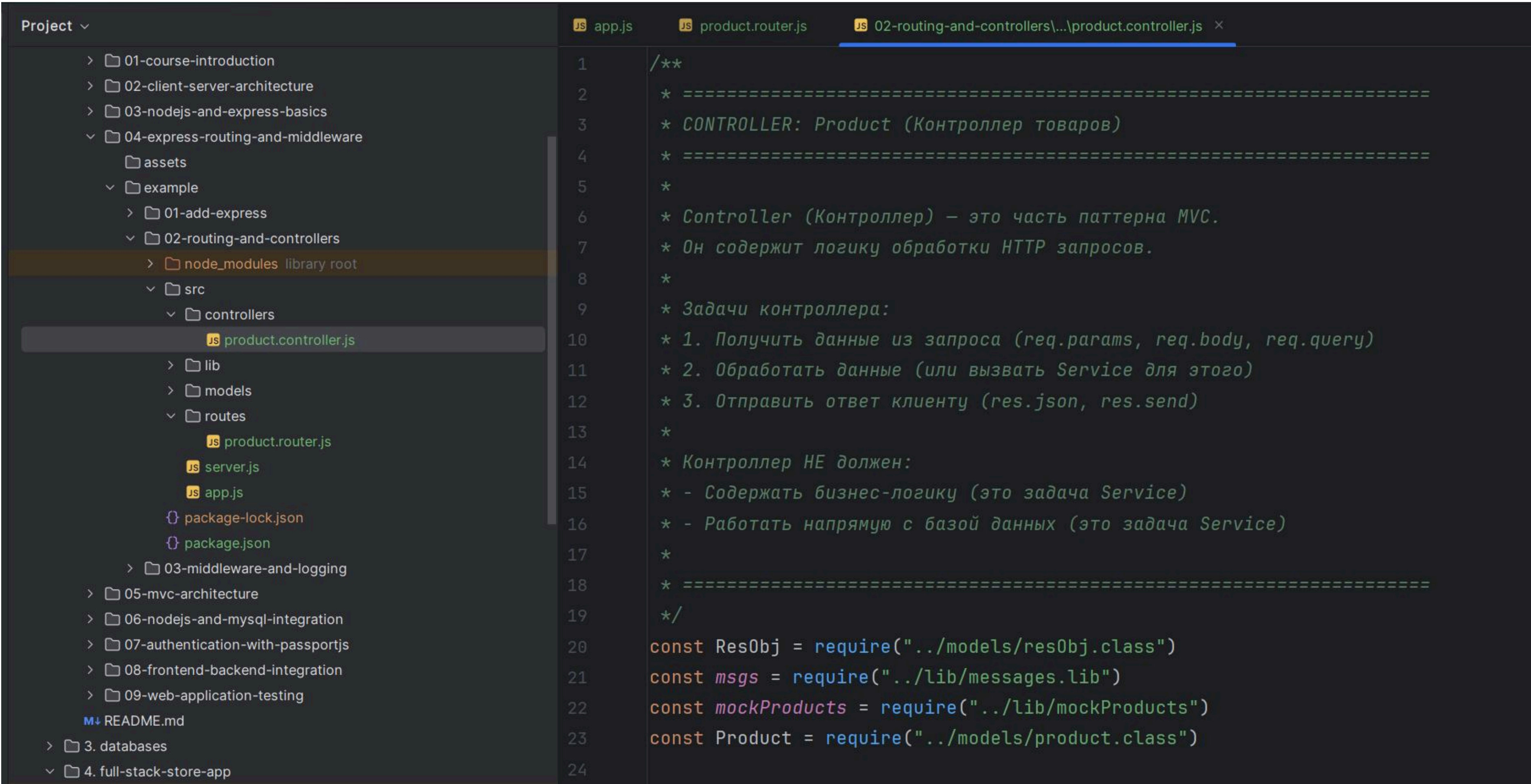
51

module.exports = productRouter;

52

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

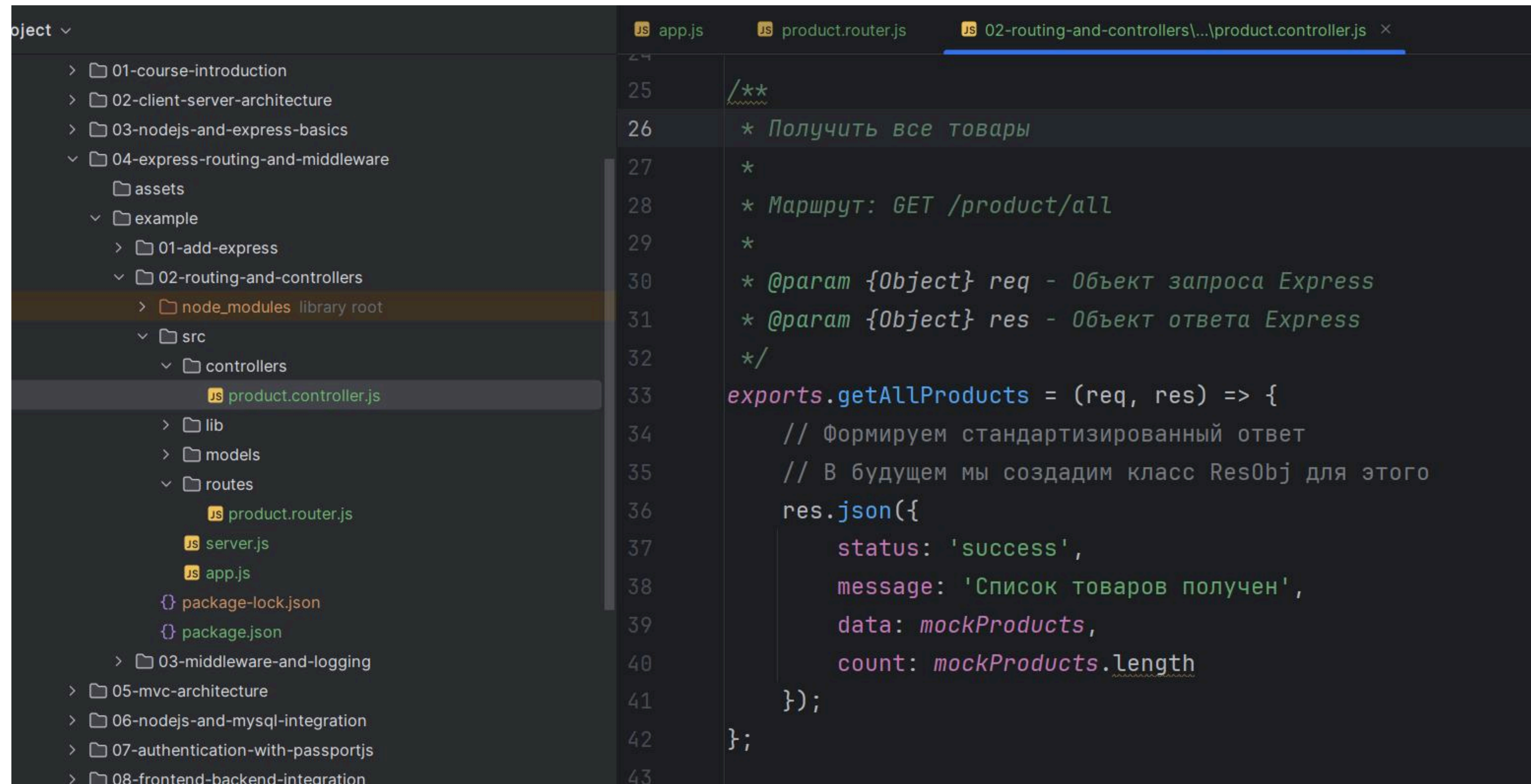
ПРАКТИКА



ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

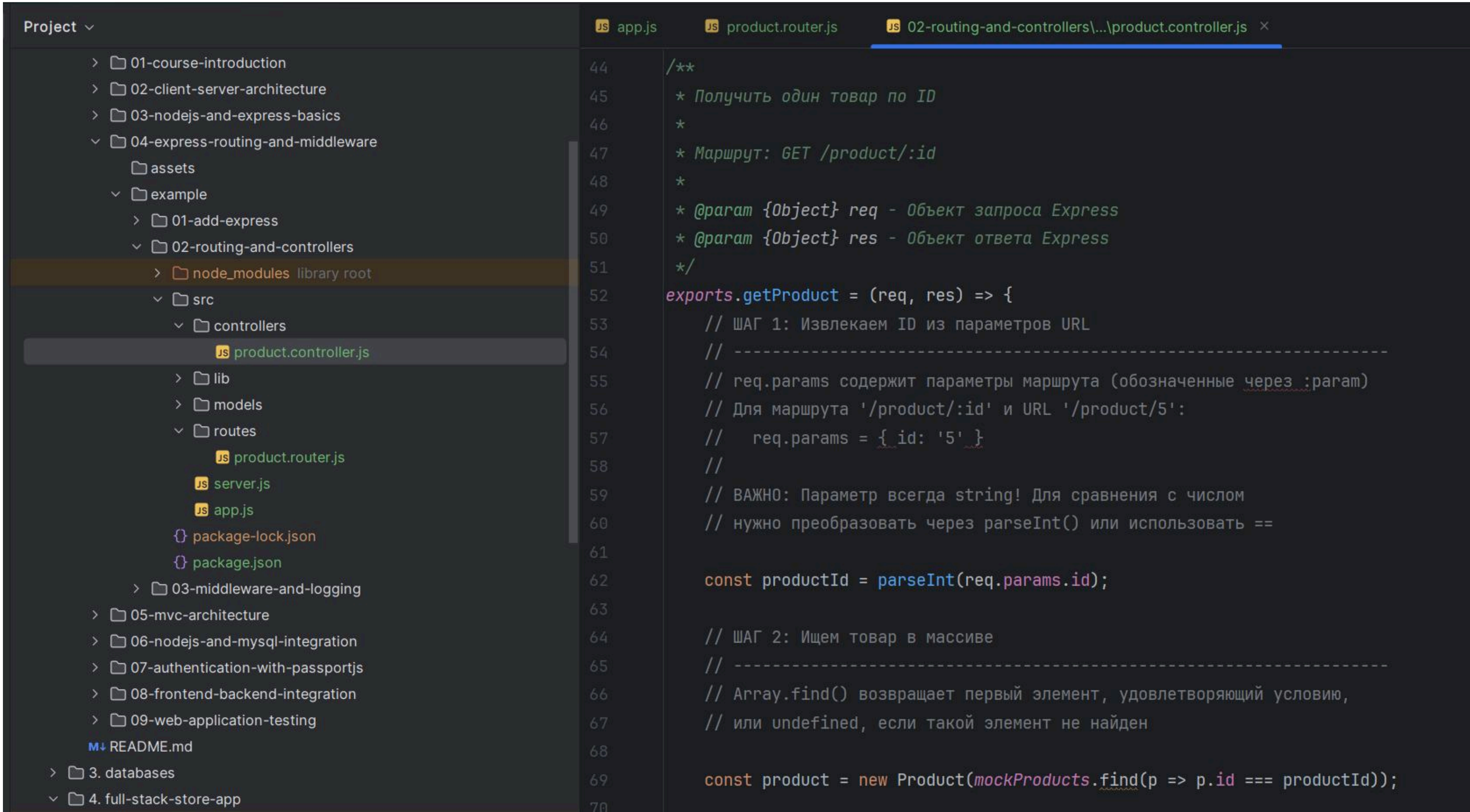


```
project ▾
  > 01-course-introduction
  > 02-client-server-architecture
  > 03-nodejs-and-express-basics
  ▾ 04-express-routing-and-middleware
    assets
    ▾ example
      > 01-add-express
      ▾ 02-routing-and-controllers
        > node_modules library root
        ▾ src
          ▾ controllers
            JS product.controller.js
          lib
          models
          ▾ routes
            JS product.router.js
            JS server.js
            JS app.js
            package-lock.json
            package.json
        > 03-middleware-and-logging
  > 05-mvc-architecture
  > 06-nodejs-and-mysql-integration
  > 07-authentication-with-passportjs
  > 08-frontend-backend-integration

JS app.js JS product.router.js JS 02-routing-and-controllers\...\product.controller.js ×
24
25 /**
26 * Получить все товары
27 *
28 * Маршрут: GET /product/all
29 *
30 * @param {Object} req - Объект запроса Express
31 * @param {Object} res - Объект ответа Express
32 */
33 exports.getAllProducts = (req, res) => {
34   // Формируем стандартизированный ответ
35   // В будущем мы создадим класс ResObj для этого
36   res.json({
37     status: 'success',
38     message: 'Список товаров получен',
39     data: mockProducts,
40     count: mockProducts.length
41   });
42 };
43
```

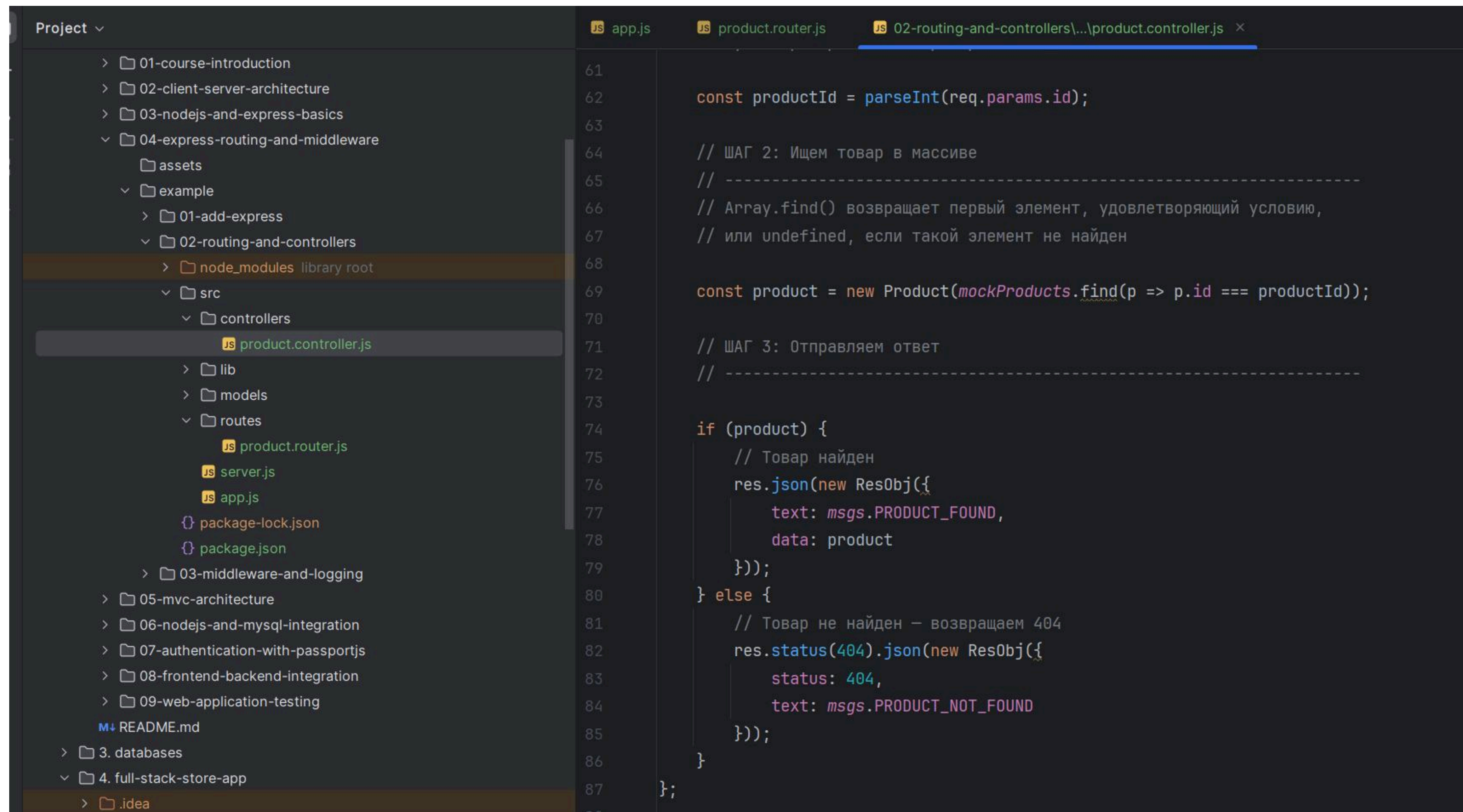

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



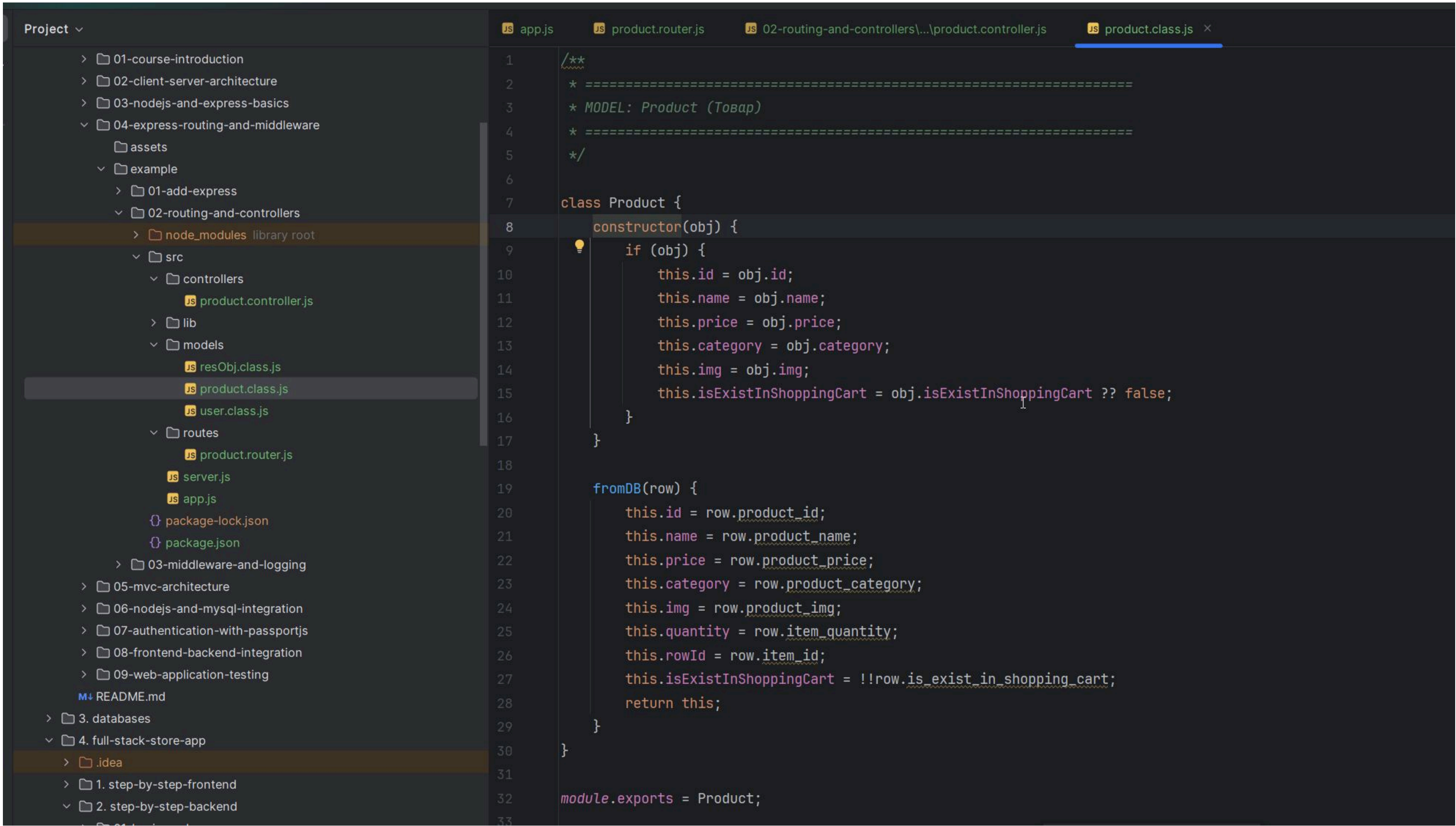
РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



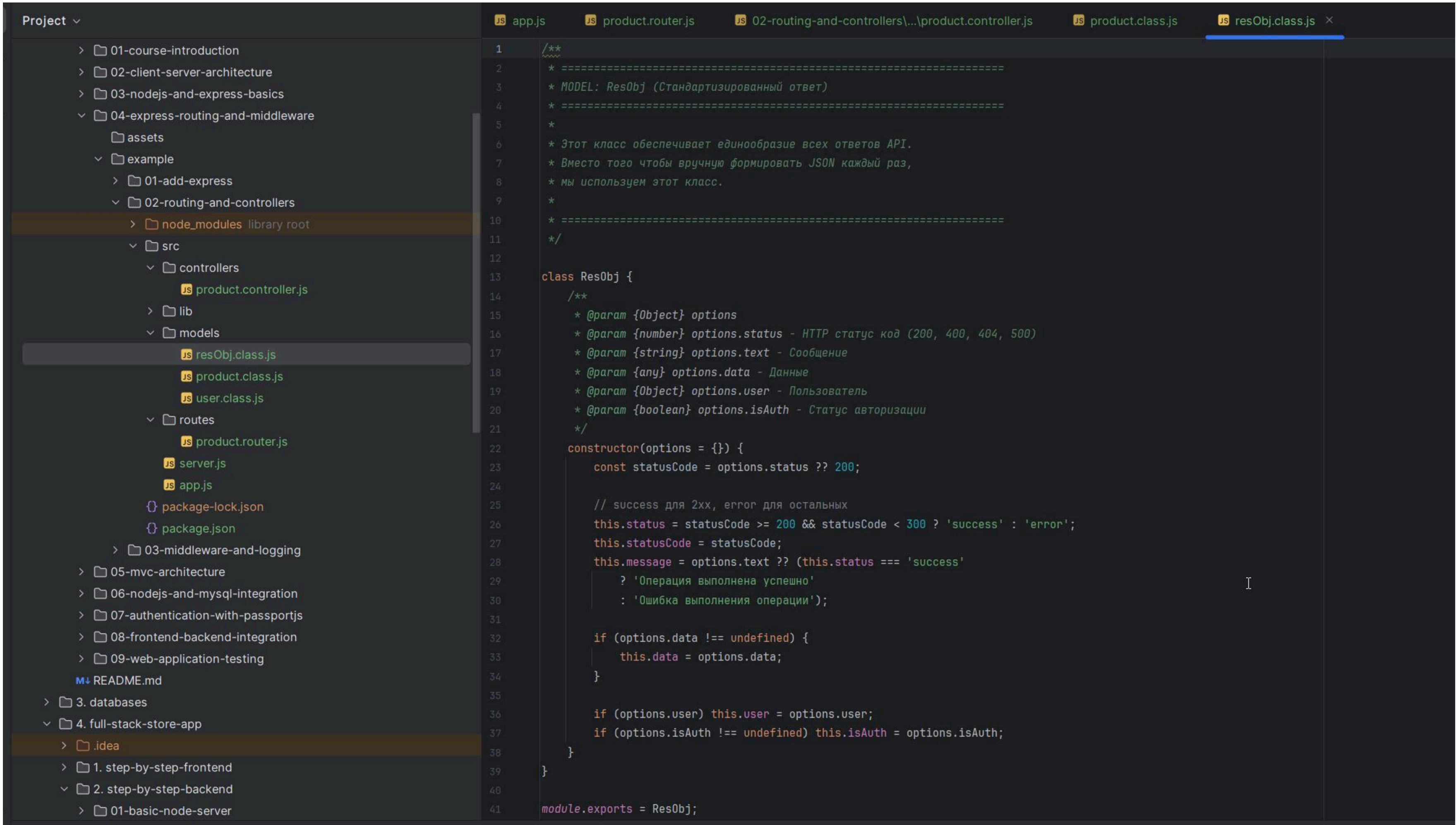
РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



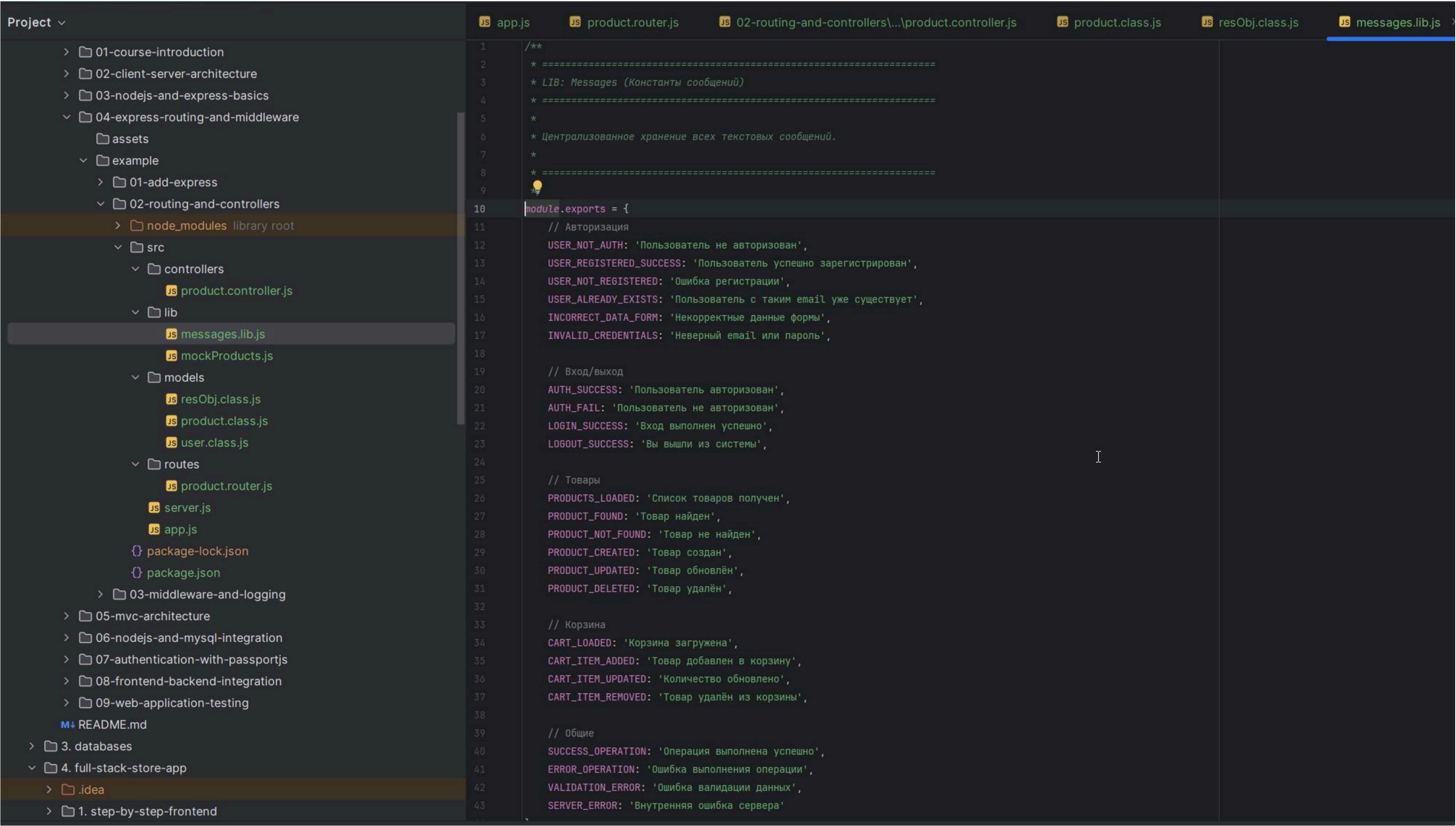
РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА



ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ

РЕАЛИЗАЦИЯ ПАТТЕРНА MVC

ПРАКТИКА

Project

> 01-course-introduction

> 02-client-server-architecture

> 03-nodejs-and-express-basics

> 04-express-routing-and-middleware

assets

example

01-add-express

02-routing-and-controllers

node_modules library root

src

controllers

product.controller.js

lib

messages.lib.js

mockProducts.js

models

resObj.class.js

product.class.js

user.class.js

routes

product.router.js

server.js

app.js

package-lock.json

package.json

> 03-middleware-and-logging

> 05-mvc-architecture

> 06-nodejs-and-mysql-integration

> 07-authentication-with-passportjs

app.js

product.router.js

02-routing-and-controllers\...\product.controller.js

product.class.js

resObj.class.js

messages.lib.js

mockProducts.js

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

/**

=====

* LIB: Mock Products (Тестовые данные)

* =====

*/

const mockProducts = [

{ id: 1, name: 'Intel Core i9-13900K', price: 55990, category: 'Процессоры', img: null },

{ id: 2, name: 'AMD Ryzen 9 7950X', price: 62990, category: 'Процессоры', img: null },

{ id: 3, name: 'NVIDIA GeForce RTX 4090', price: 159990, category: 'Видеокарты', img: null },

{ id: 4, name: 'AMD Radeon RX 7900 XTX', price: 89990, category: 'Видеокарты', img: null },

{ id: 5, name: 'ASUS ROG STRIX Z790-E', price: 42990, category: 'Материнские платы', img: null },

{ id: 6, name: 'MSI MAG B650 TOMAHAWK', price: 28990, category: 'Материнские платы', img: null },

{ id: 7, name: 'Corsair Vengeance DDR5 32GB', price: 15990, category: 'Оперативная память', img: null },

{ id: 8, name: 'G.Skill Trident Z5 RGB 64GB', price: 29990, category: 'Оперативная память', img: null },

{ id: 9, name: 'Samsung 990 PRO 2TB NVMe', price: 18990, category: 'Накопители', img: null },

{ id: 10, name: 'WD Black SN850X 1TB', price: 12990, category: 'Накопители', img: null },

{ id: 11, name: 'Corsair RM1000x 1000W', price: 19990, category: 'Блоки питания', img: null },

{ id: 12, name: 'Be Quiet! Dark Power 13 850W', price: 16990, category: 'Блоки питания', img: null }

];

module.exports = mockProducts;

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?